

克丽丝腾的实验室

基于 Qt/C++ 的重力反转解谜游戏作业报告

报告名称	作业报告
组号	113
组名	不敢再玩卫戍协议了
组长	许耀中
组员	李春
项目类型	Qt Widgets 桌面端二维物理解谜游戏

本报告围绕《克丽丝腾的实验室》的程序功能、游戏机制、模块与类设计、小组分工以及项目总结反思进行说明。项目以“重力方向切换”为核心玩法，通过 Qt 图形场景、JSON 关卡系统、可视化关卡设计师、本地存档与背景音乐，构成一个完整的闯关制物理解谜游戏。

报告内容在 README 和项目架构分析基础上进一步扩展，重点说明程序如何运行、各模块如何协作、关卡设计师为何是项目亮点、GitHub 协同开发如何进行、资源部署如何完成，以及开发过程中遇到的不足与改进方向。

本项目既关注最终可玩的游戏效果，也关注课程项目中的工程完整性。除了主流程闯关之外，项目还提供了自定义关卡、关卡校验、试玩机制和本地进度保存，使其更接近一个完整的小型桌面游戏。

一、程序功能介绍

1.1 游戏背景与设计目标

《克丽丝腾的实验室》是一款基于物理法则重构的极简风格闯关解谜游戏。游戏背景设定在一个实验室空间中，玩家并不是传统意义上的角色操控者，而是实验室中的“重力调试员”。玩家需要通过改变空间中的重力方向，引导主角小球在复杂的实验关卡中移动，避开危险区域，触发机关，收集关键物品，最终抵达实验终点。

与传统平台动作游戏不同，本游戏不允许玩家直接控制小球的左右移动。小球的运动主要由当前重力方向和地图环境共同决定。玩家能够直接控制的是空间重力方向，包括上、下、左、右四个方向。每一次重力切换都会改变小球受力方向，从而改变小球的运动轨迹。

这种玩法使游戏核心不再是简单的移动和跳跃，而是对空间方向、路径规划、机关顺序和物理反馈的综合判断。玩家需要思考何时切换重力、切换到哪个方向、小球会撞到哪里、是否会进入死亡区、是否需要先拿钥匙或通过传送门等。

项目的设计目标包括三点：第一，实现一个可正常运行的 Qt 桌面端解谜游戏；第二，实现 JSON 数据驱动的关卡系统，使关卡内容可以独立于代码扩展；第三，提供一个可视化关卡设计师，使关卡制作、校验和试玩形成闭环。

1.2 核心玩法与操作方式

游戏的核心机制是“重力反转”。玩家通过键盘上的 W、A、S、D 改变重力方向，小球会沿当前重力方向运动，直到遇到墙体、机关或其他地图元素。玩家需要利用墙体、缓冲区、蹦床、传送门、钥匙、门、激光门等元素完成关卡。

按键	功能	说明
W	切换重力向上	小球受到向上的重力影响，适合脱离下方危险区域或进入上方通道。
A	切换重力向左	小球向左侧移动，可用于贴近左墙或进入左侧机关区域。
S	切换重力向下	小球向下运动，是默认方向中最容易理解的一种。
D	切换重力向右	小球向右侧移动，常用于推进关卡路线。

游戏界面会实时显示当前关卡、游戏时间、当前重力方向、重力反转次数、死亡次数和通关状态。玩家需要根据这些反馈判断当前操作是否合理，并不断调整重力方向完成解谜。部分关卡还会设置目标重力反转次数，用于评价玩家是否以较高效率完成关卡。

1.3 程序主要功能

功能模块	具体说明
主菜单与入口	游戏启动后进入主菜单，玩家可以选择开始游戏、关卡选择、关卡设计师、操作说明或退出游戏。
核心游戏场景	通过 QGraphicsScene / QGraphicsView 绘制地图，处理小球显示、重力切换、物理移动、碰撞检测、道具触发和胜负判断。
关卡系统	使用 JSON 文件描述关卡，支持内置关卡和自定义关卡，程序启动时自动读取 levels/ 与 custom_levels/ 中的合法关卡。
关卡设计师	内置可视化关卡编辑器，支持图形化创建、编辑、校验、试玩和保存关卡，无需手写 JSON 字符串。
进度存档	通过 save/progress.json 保存通关状态、最佳星级、最短时间、最少反转次数、死亡次数和关卡解锁进度。
背景音乐	通过 AudioPlayer 播放 music/ 文件夹中的外部音频资源，增强游戏氛围。

二、游戏机制与地图元素

2.1 地图元素编码

关卡地图使用字符编码表示，每个字符代表一种地图元素。该设计让关卡可以通过 JSON 文件保存，便于加载、编辑和扩展。关卡设计师中看到的是可视化网格，底层保存时会转换成字符矩阵。

字符	含义	作用
0	空地	小球可以通过的普通区域。
1	墙体	阻挡小球移动，是基础碰撞元素。
2	起点	小球出生位置，每个关卡必须且只能有一个。
3	终点	小球到达后触发通关。

字符	含义	作用
4	死亡区	小球进入后死亡并重置。
6	缓冲区	减缓或改变小球运动反馈，使关卡节奏更丰富。
8	数据碎片	可收集元素，用于增加关卡目标感。
L	激光门	形成动态或条件式障碍。
9/u/v/w/r/l	多方向蹦床	将小球弹向右上、左上、右下、左下、右、左等方向。
K / A	钥匙与门	小球收集钥匙后可以通过门。
B	传送门	成对出现，用于空间传送。
5 / 7	旧版元素	弹射块、传送带等历史保留元素，目前主要作为兼容代码存在。

2.2 地图元素组合方式

不同地图元素可以组合出不同类型的谜题。例如，墙体用于限制运动空间，死亡区用于制造风险，钥匙和门用于设计前置目标，传送门用于改变空间连接，蹦床用于改变运动方向，激光门用于增加时机判断。

由于玩家不能直接控制小球左右移动，关卡设计时需要考虑小球在不同重力方向下的落点。一个看似简单的墙体位置，也可能决定玩家是否能顺利进入下一条通路。因此本项目的关卡设计不只是摆放障碍，更像是在设计一套基于重力反馈的路径推理题。

2.3 JSON 关卡格式

关卡 JSON 文件包含关卡名称、目标反转次数、地图数据和玩家可编辑候选点等字段。地图数据使用字符串数组表示，每个字符串代表地图的一行。

```
{
  "name": "基础关",
  "targetReverseCount": 6,
  "map": [
    "111111111111",
    "120000000001",
    "100000000001",
    "100400000031",
    "111111111111"
  ],
  "editablePoints": []
}
```

这种数据格式具有较好的可读性和扩展性。开发者可以直接查看 JSON 文件理解关卡结构，关卡设计师也可以将编辑结果保存成相同格式，从而保证编辑器、游戏逻辑和关卡文件之间保持一致。

三、关卡设计师功能设计

3.1 功能定位

关卡设计师是本项目的重要亮点之一。它相当于内置的可视化关卡编辑器，玩家或开发者可以不直接修改代码，而是在图形界面中设计新的实验关卡。该模块使项目从“固定关卡游戏”扩展为“可持续创作关卡的游戏”。

关卡设计师主要面向关卡创作和调试。用户可以设置关卡名称、地图宽度、地图高度和目标重力反转次数，并在表格化地图编辑区域放置墙体、起点、终点、死亡区、缓冲区、数据碎片、钥匙、门、传送门、激光门和不同方向的蹦床等元素。

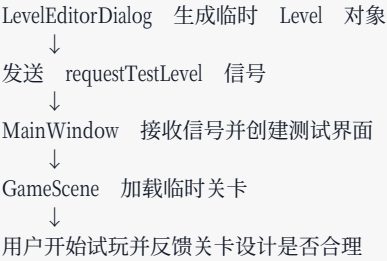
3.2 主要功能

功能	说明
新建关卡	创建一个新的空白关卡，并设置地图尺寸和关卡名称。
地图可视化编辑	用表格展示地图，每个格子代表一个地图块，用户点击格子即可放置元素。
地图元素选择	提供空地、墙体、起点、终点、死亡区、缓冲区、数据碎片、钥匙、门、传送门、激光门、多方向蹦床等工具。
自动生成边框墙	快速搭建封闭实验空间，减少重复操作，并避免小球离开地图范围。
清空地图	一键清除当前地图内容，方便重新设计。
撤销与重做	降低误操作带来的影响，提高关卡制作效率。
导入 JSON	导入已有 JSON 关卡文件，在其基础上继续修改。

功能	说明
关卡校验	检查起点、终点、门钥匙、传送门配对、未知字符和候选点位置。
试玩当前关卡	无需先保存即可进入游戏场景测试关卡。
保存关卡	将设计好的关卡保存到内置关卡目录或自定义关卡目录。

3.3 编辑器与游戏场景的连接

关卡设计师并不是孤立的 UI 工具，它会与 Level 数据结构、LevelManager 校验逻辑、MainWindow 界面切换逻辑和 GameScene 临时关卡试玩逻辑联动。当用户点击试玩时，编辑器会把当前地图转换为临时 Level 对象，再通过信号传递给主窗口，主窗口创建测试游戏界面，由 GameScene 加载临时关卡运行。



该设计使关卡制作形成“编辑 - 校验 - 试玩 - 修改 - 保存”的闭环，显著降低了关卡迭代成本。如果没有该功能，开发者需要手写 JSON、重新加载游戏、再观察关卡效果，效率会低很多。

3.4 玩家可编辑候选点

关卡设计师还支持设置玩家可编辑候选点。该功能允许关卡设计者指定部分特殊格子，使这些位置在特定模式下可以被玩家调整或选择。它为关卡提供了更高的灵活性，也让关卡具备一定的二次解谜空间。

例如，设计者可以限定几个可调整位置，让玩家在固定规则下补全路线或放置模块。这样关卡不再只是固定路线解谜，还可以加入“玩家参与构造”的策略性。

四、项目总体架构与目录设计

4.1 总体架构

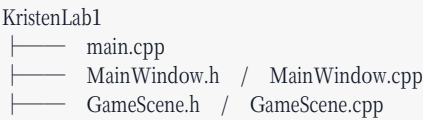
项目整体采用 Qt Widgets 桌面应用架构。程序入口为 main.cpp，主界面由 MainWindow 管理，核心游戏逻辑由 GameScene 负责。关卡数据、进度存档、音频播放和关卡编辑等功能由不同类承担。



4.2 分层结构

层次	负责内容
程序入口层	由 main.cpp 创建 QApplication 和 MainWindow，启动 Qt 事件循环。
界面控制层	由 MainWindow 和 LevelEditorDialog 负责主菜单、游戏窗口、关卡选择和关卡设计师界面。
游戏逻辑层	由 GameScene 处理小球移动、重力切换、碰撞检测、机关触发和胜负判断。
数据管理层	由 LevelManager 和 ProgressManager 处理关卡数据与玩家进度。
资源辅助层	由 Ball、AudioPlayer、tiledefs.h、constants.h 等模块提供对象、音频、地图定义和常量支持。

4.3 项目目录



```
├── LevelManager.h / LevelManager.cpp
├── LevelEditorDialog.h / .cpp
├── ProgressManager.h / .cpp
├── Ball.h / Ball.cpp
├── AudioPlayer.h / .cpp
├── tiledefs.h / constants.h
├── levels/ custom_levels/
├── music/ save/
├── resources/images/ resources.qrc
└── CMakeLists.txt
```

这种目录组织方式将界面、游戏逻辑、关卡数据、存档和资源管理分开，有利于后续维护。虽然核心类仍然较大，但整体工程结构已经比较清楚。

五、核心模块与类设计

5.1 main.cpp 与 MainWindow

main.cpp 是程序入口，负责创建 QApplication、创建 MainWindow、显示主窗口并进入 Qt 事件循环。Qt 程序依赖事件循环处理窗口显示、鼠标点击、键盘输入、定时器刷新等事件，因此 QApplication 的创建和执行是必要步骤。

MainWindow 是项目主窗口类，也是界面控制中心。它负责初始化主窗口、创建主菜单、响应开始游戏、关卡选择、关卡设计师等按钮、创建 GameScene 并放入 QGraphicsView 显示、更新游戏状态栏、显示当前键盘方向提示、打开帮助说明、打开关卡设计师，并在试玩关卡时接收编辑器传入的临时关卡对象。

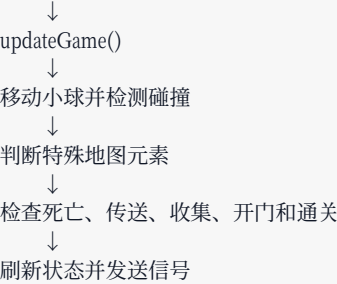
MainWindow 和 GameScene 之间主要通过信号与槽机制通信。GameScene 在游戏状态变化时发出信号，例如当前关卡、时间、重力方向、反转次数、死亡次数等变化，MainWindow 接收信号后更新界面显示。这样 MainWindow 不需要直接参与小球运动和碰撞检测，降低了界面层与游戏逻辑层的耦合。

5.2 GameScene

GameScene 是项目中最核心的类，继承自 QGraphicsScene。它承担游戏运行时的大部分逻辑，是地图绘制、物理移动、碰撞检测、机关触发和胜负判断的控制中心。

职责类别	具体内容
关卡加载与地图绘制	加载当前关卡，根据地图数据绘制墙体、起点、终点、死亡区、机关和道具。
小球与输入	创建并管理小球对象，处理 W/A/S/D 键盘输入，切换重力方向并记录反转次数。
运动与碰撞	根据当前重力方向移动小球，检测墙体、门、激光门等阻挡元素。
特殊元素效果	处理死亡区、终点、钥匙、门、传送门、蹦床、缓冲区、激光门和数据碎片。
状态与存档	判断通关或死亡，更新游戏时间、死亡次数、通关状态，并调用进度管理逻辑。
试玩支持	加载关卡设计师生成的临时 Level 对象，实现快速测试。

QTimer 定时触发



GameScene 的优点是逻辑集中，便于快速完成可玩的游戏闭环。缺点是职责较多，后续如果继续维护，可以将地图渲染、物理控制、碰撞检测、Tile 效果系统和游戏状态管理进一步拆分。

5.3 Level 与 LevelManager

Level 是关卡数据结构，用于保存关卡名称、地图数据、目标重力反转次数、玩家可编辑候选点、是否为自定义关卡和关卡来源文件路径。它是 JSON 文件和游戏运行之间的桥梁。

LevelManager 负责关卡数据管理，包括扫描内置关卡目录、扫描自定义关卡目录、读取 JSON 文件、将 JSON 转换为 Level 对象、保存 Level 为 JSON、校验关卡是否合法、标记关卡来源，并在内置关卡缺失时提供备用关卡。

5.4 关卡校验规则

校验项	意义
地图不能为空	避免加载空关卡导致无法绘制或运行。
每一行宽度一致	保证二维地图矩阵规则，便于定位格子。
有且只有一个起点	确保小球出生位置唯一。
至少一个终点	确保关卡可以通关。
门必须搭配钥匙	避免门机制无法完成。
传送门成对出现	避免进入传送门后无法确定出口。
不能出现未知字符	保证游戏逻辑能识别所有地图元素。
候选点不能覆盖关键格子	避免玩家编辑点破坏起点、终点、墙体、门、钥匙、传送门等规则。

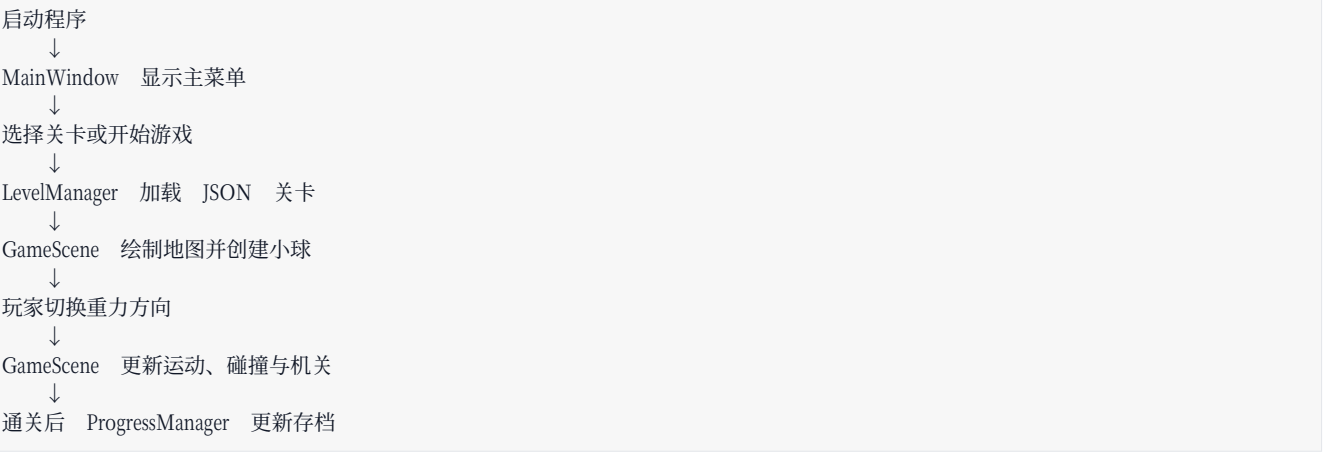
5.5 其他辅助类

模块	说明
ProgressManager	读取和写入 save/progress.json，记录通关状态、最佳星级、最短时间、最少反转、死亡次数和最高解锁关卡。
Ball	封装小球对象，包括图形显示、当前位置和碰撞区域，使 GameScene 可以围绕小球进行运动和碰撞处理。
AudioPlayer	封装 miniaudio，负责初始化音频引擎、加载背景音乐、播放、停止、设置音量 and 处理路径问题。
tiledefs.h	集中定义地图元素字符和含义，保证编辑器、校验器和游戏逻辑对 tile 的理解一致。
constants.h	保存格子尺寸、地图尺寸、速度参数和界面参数，减少硬编码，提高维护效率。

六、程序流程、资源部署与测试

6.1 游戏运行流程

游戏运行过程可以概括为：程序启动后进入主菜单，玩家选择开始游戏或指定关卡，MainWindow 创建 GameScene，GameScene 通过 LevelManager 加载关卡并绘制地图，随后定时器驱动游戏循环，小球在重力和地图元素作用下运动，最终根据是否到达终点或进入死亡区判断状态。



6.2 背景音乐与构建步骤

项目使用 CMake 构建，并支持 Qt5 或 Qt6。图片和动画资源通过 resources.qrc 编译进 Qt 资源系统，程序运行时可以通过 Qt 资源路径访问。关卡 JSON 文件和音乐文件作为外部资源读取。

程序运行时主要依赖 levels/、custom_levels/、music/ 和 save/ 等目录。其中 levels/ 保存内置关卡，custom_levels/ 保存自定义关卡，music/ 保存背景音乐，save/ 保存玩家进度。CMake 已经包含对 levels/ 和 custom_levels/ 的复制逻辑，但 music/ 目录需要手动复制到 build 文件夹。

步骤	说明
1	先在 Qt Creator 中正常构建一次项目，生成 build 文件夹。
2	进入生成的 build 文件夹。
3	将项目目录中的 music/ 文件夹复制到 build 文件夹中。

步骤	说明
4	回到 Qt Creator，重新构建并运行项目。

6.3 测试内容

测试项	测试目标
主菜单跳转	确认开始游戏、关卡选择、关卡设计师、操作说明和退出功能正常。
关卡加载	确认内置关卡和自定义关卡可以被 LevelManager 正确读取。
重力切换	确认 W/A/S/D 能正确改变小球运动方向并记录反转次数。
碰撞检测	确认小球不会穿过墙体、门、激光门等阻挡元素。
特殊元素	确认死亡区、终点、钥匙、门、传送门、蹦床、缓冲区等效果正常。
关卡设计师	确认编辑器可以新建、导入、校验、试玩和保存关卡。
存档更新	确认通关后 progress.json 中的记录能够正确更新。
背景音乐	确认复制 music/ 文件夹后程序可以正常播放背景音乐。

测试过程中主要采用手动运行测试和关卡验证测试。虽然没有建立完整自动化测试体系，但通过多轮运行和关卡试玩，保证了主要流程能够正常完成。

七、小组成员分工与 GitHub 协同

7.1 小组基本信息

项目	内容
项目名称	克丽丝腾的实验室
组号	113
组名	不敢再玩卫戍协议了
组长	许耀中
组员	李春

7.2 按功能划分的具体分工

功能/模块	许耀中负责内容	李春负责内容
整体架构与工程搭建	负责 Qt/CMake 工程结构规划、主要类之间的调用关系、构建流程和最终整合。	参与工程结构讨论，配合整理界面资源、运行测试和文档说明。
主菜单与页面切换	提供游戏启动流程和 GameScene 创建逻辑支持。	负责 MainWindow 主菜单、按钮交互、开始游戏、关卡选择、帮助说明和关卡设计师入口。
核心游戏逻辑	负责 GameScene 主循环、重力方向切换、小球移动、碰撞检测、死亡与通关判断。	参与测试游戏体验，反馈操作提示、界面状态显示和玩家可理解性问题。
地图元素与机关	负责钥匙、门、传送门、死亡区、终点、蹦床、缓冲区、激光门等核心逻辑实现。	负责地图元素显示效果、素材整理和不同元素在界面上的可识别性调整。
关卡系统	负责 Level 数据结构、LevelManager 关卡加载、保存、校验和自定义关卡支持。	参与设计关卡格式说明，测试内置关卡和自定义关卡是否能正确加载。
关卡设计师	负责编辑器与 Level/GameScene 的数据衔接、关卡校验、试玩逻辑和保存流程。	负责 LevelEditorDialog 界面布局、地图网格编辑交互、工具选择、撤销重做体验和可用性测试。
存档系统	负责 ProgressManager、progress.json 读写、通关记录和解锁进度更新。	参与验证通关后界面反馈和关卡解锁显示是否符合预期。
背景音乐	负责 AudioPlayer 接入、miniaudio 播放逻辑和 music 文件夹读取说明。	负责音乐资源整理，协助测试复制 music 文件夹后的播放效果。
关卡设计与测试	负责设计部分测试关卡，验证机关逻辑和边界情况。	负责从玩家视角试玩关卡，调整难度、路径可读性和界面提示。
文档与最终提交	负责 README、作业报告中后端架构、构建运行、核心逻辑部分说明。	负责 README、作业报告中界面功能、关卡设计师、使用体验和分工内容整理。

7.3 GitHub 协同开发方式

本项目开发过程中使用 GitHub 进行协同管理。小组将项目代码托管在 GitHub 仓库中，通过提交记录保存每次功能修改，便于追踪代码变化、回退错误修改和进行最终整合。

协作时，许耀中主要提交后端逻辑、GameScene、LevelManager、ProgressManager、AudioPlayer 和 CMake 相关修改；李春主要提交 MainWindow、LevelEditorDialog、界面交互、资源整理和关卡设计相关修改。两位成员在完成各自功能后，将代码推送到 GitHub，再进行同步、合并和本地运行测试。

GitHub 在本项目中的作用主要包括：第一，保存代码版本，避免文件互相覆盖；第二，记录每个阶段完成的功能；第三，便于两位成员在不同电脑上同步最新代码；第四，在整合阶段通过提交历史定位问题来源；第五，辅助完成最终打包、README 和报告整理。

协同环节	具体做法
代码托管	将完整 Qt/C++ 工程上传到 GitHub 仓库，统一管理源码、关卡、资源说明和文档。
功能提交	完成一个较明确的功能后进行 commit，例如关卡加载、主菜单界面、关卡设计师、存档系统或音乐接入。
同步更新	开发前先拉取仓库最新内容，避免基于旧代码继续修改。
冲突处理	当界面文件、配置文件或文档同时修改时，由两人沟通后手动合并，确保最终版本可构建运行。
联调测试	合并后在 Qt Creator 中重新构建运行，检查主菜单、关卡、编辑器、音乐和存档是否正常。
文档整理	README 和作业报告内容根据 GitHub 中最终项目结构和功能实现统一整理。

通过 GitHub 协同，项目开发过程更加清晰。即使某个功能修改导致运行问题，也可以通过提交记录查看最近改动，定位是游戏逻辑、界面交互、资源路径还是配置文件造成的问题。

7.4 协作方式总结

本小组采用“核心逻辑与界面表现并行推进”的方式。许耀中主要保证游戏规则、数据流和后端逻辑稳定，李春主要保证界面易用性、视觉表现和编辑器操作体验。项目后期两位成员共同进行联调，包括测试关卡加载、小球运动、机关碰撞、编辑器保存、存档更新和背景音乐播放等功能。

八、项目总结与反思

8.1 项目完成情况

本项目完成了一个具备完整游戏流程的 Qt 桌面端重力解谜游戏。玩家可以从主菜单进入游戏，通过关卡选择进入指定关卡，利用 W、A、S、D 改变重力方向，引导小球完成解谜并抵达终点。项目还实现了关卡设计师、自定义关卡、关卡校验、进度存档和背景音乐播放等功能。

从功能完整性来看，项目不仅具备一个游戏应有的开始、游玩、通关和保存流程，还包含内容制作工具。关卡设计师的加入使项目不再依赖固定关卡，后续可以继续添加新的关卡内容。

8.2 项目收获

通过本项目，我们进一步熟悉了 Qt Widgets、QGraphicsScene、CMake、JSON 数据处理和面向对象程序设计。一个游戏项目并不只是界面或单个算法，而是多个系统协同工作的结果，需要界面入口、地图渲染、输入处理、物理反馈、碰撞检测、数据保存、资源读取和错误处理等多个部分共同配合。

本项目的重要收获之一是理解了数据驱动的意义。通过 JSON 文件保存关卡，游戏内容可以独立于代码进行扩展。配合关卡设计师后，关卡制作过程更加直观，也更容易测试和修改。

另一个重要收获是理解了工具开发和 GitHub 协同的重要性。关卡设计师极大提高了内容制作效率；GitHub 则帮助我们管理代码版本、同步开发进度、避免文件覆盖，并在整合阶段提供问题追踪依据。

8.3 存在的问题

问题	说明
核心类职责较重	GameScene 承担地图绘制、物理移动、碰撞检测、机关触发、胜负判断等大量逻辑；LevelEditorDialog 也承担界面搭建、地图编辑、撤销重做、导入导出、校验和试玩等功能。
资源部署不够自动化	背景音乐需要手动复制 music/ 文件夹到 build 目录。若用户忘记复制，游戏仍可运行，但无法正常播放音乐。
历史元素未完全统一	项目中存在弹射块、传送带以及部分预留 tile 编码，代码、编辑器说明和校验逻辑可能不完全一致。

问题	说明
关卡设计师仍可增强	当前已支持基础编辑、保存、导入、校验和试玩，但还可以增加缩略图、自动通路检测、地图模板、机关参数配置和更友好的错误提示。
协作流程仍可规范	虽然使用了 GitHub，但后续可以进一步规范分支命名、commit 信息、Pull Request 评审和 issue 任务拆分。

8.4 后续改进方向

后续可以将 GameScene 拆分为地图渲染模块、物理控制模块、碰撞检测模块、Tile 效果模块和游戏状态模块；将 LevelEditorDialog 拆分为编辑器 UI、地图数据模型、JSON 导入导出、撤销重做系统和校验系统。

资源方面，可以在 CMake 中加入 music/ 文件夹自动复制逻辑，使背景音乐资源和关卡资源一样能够自动部署到运行目录。协作方面，可以使用更规范的 GitHub 工作流，例如每个功能创建单独分支，合并前进行运行测试，并通过 issue 记录 bug 和优化项。

关卡设计师方面，可以增加关卡预览、地图模板、自动检测、错误定位和更友好的 UI 提示。界面方面，可以继续优化按钮样式、地图块显示、背景图、动画效果和音效反馈，使游戏体验更完整。

8.5 项目总结

《克丽丝腾的实验室》以“ 玩家不能直接控制小球，而是通过改变重力方向影响小球运动 ”为核心玩法，围绕 Qt 图形场景和 JSON 关卡系统构建了一个完整的物理解谜游戏。尤其是关卡设计师的加入，使项目具有较强的扩展能力，不再局限于固定关卡内容。

整体来看，本项目完成了从玩法设计、程序架构、核心逻辑、界面交互、关卡编辑、资源管理、GitHub 协同到文档整理的完整过程。通过本次项目，我们提升了 Qt 桌面程序开发能力，也更加理解了游戏项目中模块设计、数据驱动、资源部署和团队协作的重要性。

九、补充说明：关键流程与设计取舍

9.1 为什么选择 JSON 关卡

项目选择 JSON 作为关卡格式，是因为 JSON 结构清晰、便于人工阅读，也便于 Qt 程序解析。相比把地图直接写在 C++ 代码中，JSON 关卡能够让程序逻辑和关卡内容解耦。这样做的好处是：新增关卡不需要重新修改核心代码；关卡设计师可以直接导出文件；测试时可以快速替换关卡；自定义关卡可以和内置关卡使用同一套加载逻辑。

在本项目中，JSON 不只是数据保存方式，也承担了连接关卡设计师和游戏运行逻辑的桥梁作用。编辑器生成的地图最终保存为 JSON，游戏启动时再由 LevelManager 读取。这样可以保证制作关卡和运行关卡使用同一份数据格式，减少格式转换造成的问题。

9.2 为什么需要关卡校验

关卡校验看似只是辅助功能，但它直接影响游戏稳定性。假如地图没有起点，GameScene 无法生成小球；假如没有终点，玩家无法通关；假如传送门不是成对出现，传送逻辑无法确定目标；假如出现未知字符，绘制和碰撞逻辑都无法正确判断格子类型。因此，校验机制是关卡系统可靠运行的前提。

关卡校验还提升了关卡设计师的可用性。用户在保存或试玩前就能发现错误，而不是进入游戏后才遇到无法运行的问题。这种提前反馈可以减少调试时间，也能让关卡设计流程更加顺畅。

9.3 为什么保留外部音乐文件

背景音乐没有直接放入 Qt 资源系统，而是作为外部 music 文件夹读取。这样做的优点是便于替换音乐资源，不需要重新编译程序；缺点是运行前必须保证 music 文件夹位于构建目录中。当前 README 和报告中都补充了音乐部署说明，后续更好的做法是在 CMake 中添加自动复制规则。

9.4 GitHub 对项目管理的意义

GitHub 在本项目中不仅用于存放代码，也帮助小组建立了基本的软件协作流程。通过 commit 记录，成员可以看到每个功能的实现过程；通过同步仓库，可以避免版本不一致；通过合并和联调，可以把后端逻辑、前端界面、关卡文件和资源文件整合成最终版本。

从课程项目角度看，该项目同时包含了可运行程序、可扩展关卡、可视化编辑器、存档、音乐资源说明和 GitHub 协同过程，能够体现一定的软件工程意识。后续如果继续迭代，可以围绕模块拆分、自动化测试、资源自动部署和协作规范继续完善。

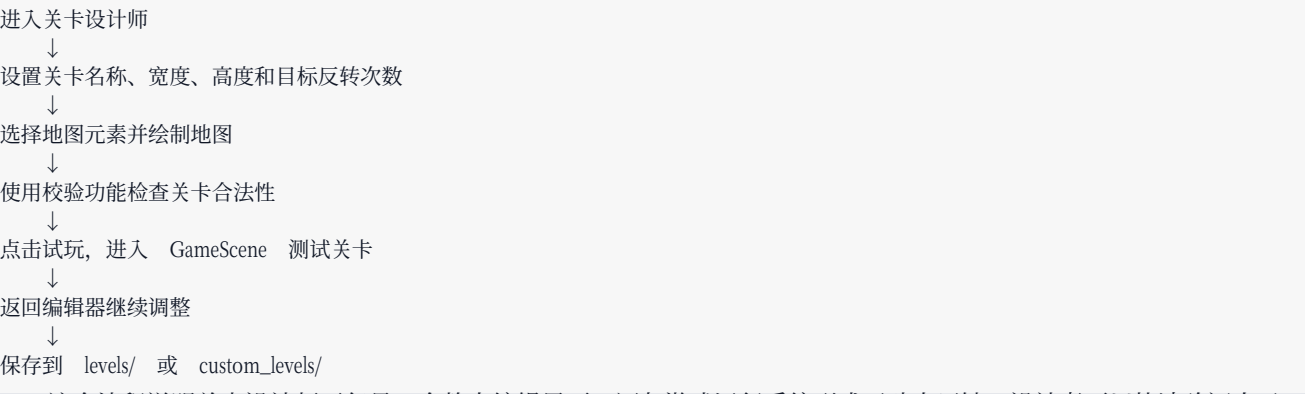
十、功能验收与典型场景补充

10.1 典型游戏场景

典型游戏流程中，玩家首先从主菜单进入关卡选择界面，选择一个已解锁关卡。程序加载对应 JSON 文件后，GameScene 根据地图字符绘制场景，小球生成在起点位置。玩家通过 W、A、S、D 改变重力方向，小球在重力作用下移动，并与墙体、机关和道具发生交互。

如果小球进入死亡区，游戏会记录一次死亡并重置小球位置；如果小球碰到钥匙，则获得开门能力；如果小球进入传送门，则传送到配对的另一个传送门；如果小球到达终点，则触发通关逻辑，并由 ProgressManager 更新存档。

10.2 关卡设计师典型使用流程



这个流程说明关卡设计师不仅是一个静态编辑界面，还与游戏运行系统形成了动态反馈。设计者可以快速验证自己的设计是否合理，并根据试玩结果继续调整地图结构。

10.3 功能验收表

验收内容	预期结果
程序启动	能够正常打开主窗口并显示主菜单。
开始游戏	能够进入默认关卡并显示游戏场景。
关卡选择	能够显示可玩关卡，并进入指定关卡。
重力控制	W/A/S/D 能正确改变小球运动方向。
地图碰撞	小球遇到墙体等阻挡元素时不能穿越。
机关交互	钥匙、门、传送门、死亡区、终点等元素能产生预期效果。
关卡设计师	能够新建、编辑、校验、试玩和保存关卡。
自定义关卡	保存到 custom_levels/ 的关卡能够被加载。
存档系统	通关后 progress.json 能记录关卡成绩。
背景音乐	复制 music/ 文件夹后能够正常播放背景音乐。
GitHub 协同	能够通过提交记录追踪成员修改，并完成最终代码整合。

上述验收内容覆盖了项目从启动、游玩、编辑、保存、协同到资源播放的核心路径。虽然项目仍有继续优化空间，但主要功能已经形成闭环，能够支撑一次完整的游戏体验和关卡创作流程。